

RWA-Vault: A Post-Quantum Anchored Protocol for Tokenised Real-World Assets with Collateralised Micro-Liquidity

Fahad Naseer Hassan Attique Fawaz Asif

Reg. 403897 Reg. 482073 Reg. 473423

Final Year Project Proposal — Complete Technical Plan

ABSTRACT

A tokenised real-world asset is a record of who owns what fraction of a physical or financial thing, and that record must stay verifiable for as long as the asset exists — commonly thirty years. Every deployed RWA protocol authenticates such records with ECDSA over secp256k1, which Shor’s algorithm recovers, so an adversary who archives signed transfer authorisations today can forge retroactive ownership once ECDLP falls. The binding engineering constraint is that the obvious fix is unaffordable: a pure-Solidity Falcon-512 verifier consumes ≈ 1.5 M gas, some $500 \times e_{\text{recover}}$. RWA-Vault therefore separates *authentication* from *anchoring*. Every intent and attestation is post-quantum signed off-chain inside a canonical envelope, and each 60-second epoch of envelopes is committed on-chain as one Keccak-256 Merkle root costing ≈ 43 k gas regardless of event count, giving every user a permanent, offline-checkable inclusion proof and driving marginal authentication cost below ECDSA past roughly fourteen events per epoch. On top of that fabric the protocol carries compliant fractional ownership (ERC-3643 with ERC-4626 vault accounting, hardened against the inflation attack with a virtual-share offset) and a lending market whose liquidation waterfall is parameterised for PKR 1,000-scale positions: a bounded reserve buffer absorbs shallow, transient health-factor breaches so a momentary price wick does not permanently cost a micro-borrower a liquidation bonus. The system is executed as a sandbox — mock PKR settlement, a simulated custodian, test-net tokens, free-tier infrastructure — behind interfaces a licensed counterparty can occupy without redesign, and every claim is measured against an otherwise-identical ECDSA-only control build selected by a single compile flag.

Index Terms—post-quantum cryptography, ML-DSA, Falcon, SLH-DSA, RWA tokenisation, ERC-3643, ERC-4626, Merkle anchoring, DeFi lending, liquidation risk.

I. SYSTEM DEFINITION

RWA-Vault is a protocol for representing verified real-world assets as permissioned on-chain ownership while allowing those positions to function as collateral for small-scale liquidity. The system separates asset authenticity, ownership, valuation and liquidity into distinct protocol states, with asset attestations and user actions authenticated using post-quantum signatures off-chain and committed through periodic Merkle anchors on-chain. ERC-3643 governs restricted ownership, ERC-4626 provides vault accounting, a five-source oracle aggregation determines collateral value, a risk engine determines borrowing and liquidation states, and a bounded reserve buffer absorbs shallow transient

breaches before liquidation. The implementation uses an EVM test network, a simulated custodian and settlement rail, and existing open-source cryptographic and smart-contract primitives; the research contribution is the integration and its cost, and it is evaluated against an otherwise-identical ECDSA-based control system.

The implementation evaluates four asset classes — gold, treasury bills, Sukuk and fractional REITs — with gold as the mandatory reference asset because it is deeply liquid, continuously priced and locally meaningful, and the other three present deliberately heterogeneous valuation, heartbeat and liquidation parameters. They exist to prove the registry and the risk engine are asset-agnostic rather than tuned to one instrument; only gold is required for the project to succeed.

Two properties, taken together, distinguish this from every deployed tokenisation platform: every authorisation and attestation in the system is signed with post-quantum cryptography and permanently anchored on-chain so that a record cannot be forged retroactively, and the credit market is parameterised for positions of the order of PKR 1,000 rather than institutional size. The document that follows is organised as a single technical path rather than as a catalogue of components. §II follows one asset and one investor from the physical world to a settled loan and states, at each transition, what changes in protocol state. §III opens each transition mathematically, in the order the flow reaches it, and carries every mechanism through the same chain: why it exists, what state it operates on, its inputs, its mathematical model, the algorithm implementing that model, where the algorithm runs, what it writes, what reads that output, how it fails, the invariant that must survive, and the experiment that tests it. §IV states only what remains after that — who is permitted to assert what, and how the system knows. §V maps every mechanism to its implementation and separates what is reused from what is adapted and what is new. §VI gives the behaviour of the protocol when each mechanism fails rather than a catalogue of threats. §VII is the experimental design against the control build, and §VIII states the contribution, its boundaries and the risks. Symbols and terms are defined once, in Appendix A, and not re-explained in the body.

II. END-TO-END PROTOCOL FLOW

This section is the spine of the proposal. One gold lot and one investor are followed from the physical vault to a settled, anchored loan; each arrow is a state transition, and each transition

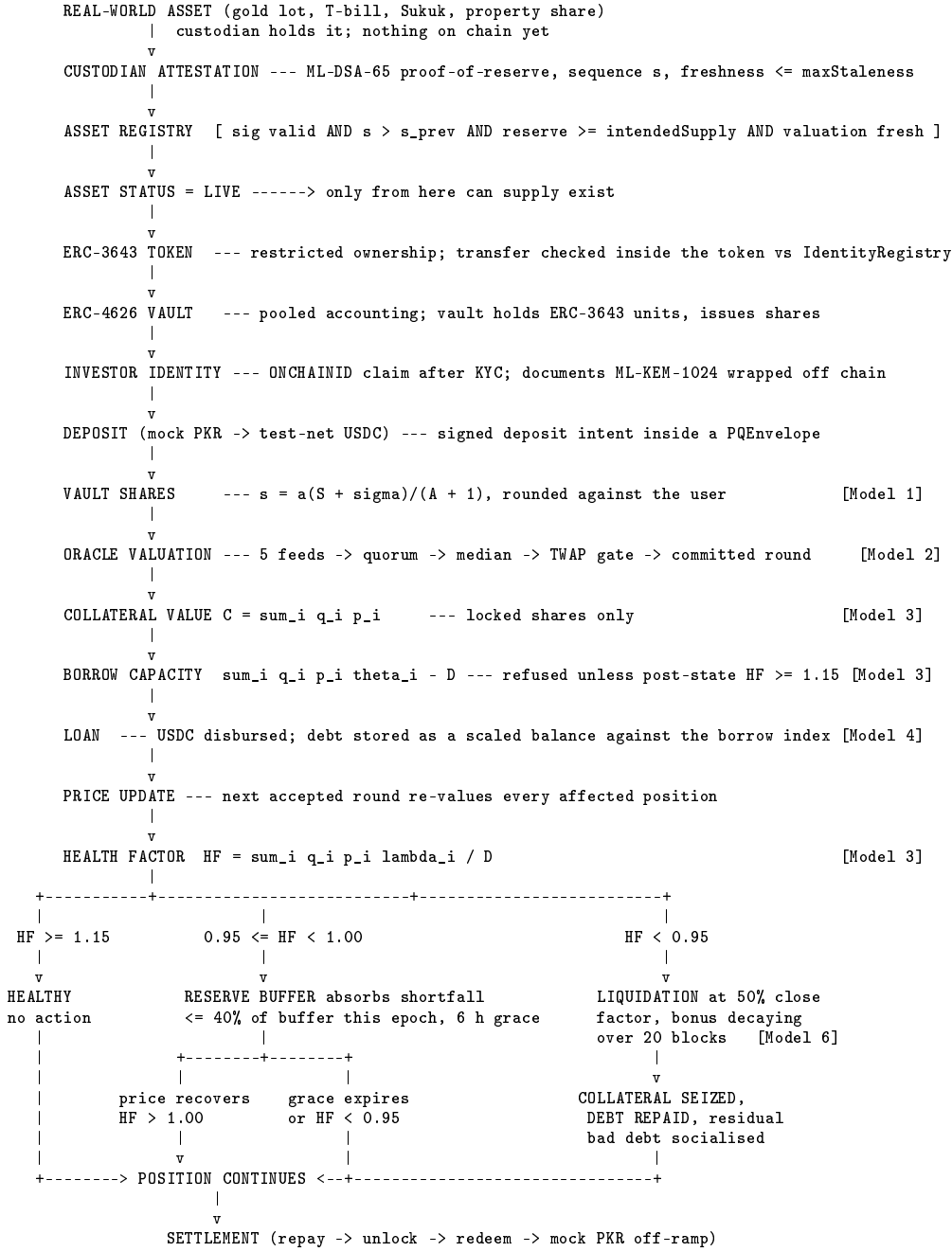


Figure 1. The asset path. Every arrow carries the condition that admits the next state; bracketed labels name the model in §III that defines the transition mathematically.

names the mechanism that §III then opens.

A. The master flow. Fig. 1 is the whole protocol in one path. Read downward: the left-hand column is what exists, and the annotation on each arrow is the condition that must hold for the next state to be reachable. Nothing in the system moves without such a condition, and every condition in the figure is a tested invariant of §VII.

Orthogonal to that path, and crossing it at every arrow that a human or a service causes, runs a second flow — the authentication fabric of Fig. 2. It is drawn separately because it is not a

stage of the asset path: it is the property that every stage of the asset path is attributable and permanently checkable.

B. Asset lifecycle. An asset is not a token; it is a state machine whose terminal live state is a precondition for any token to exist.

```

ACTION (deposit, borrow, transfer, attest, price round, admin)
|
v
PQ ENVELOPE --- canonical bytes: version, algorithm, domain,
|               chainId, contract, action, signer, nonce,
|               validFrom, validTo, payloadHash [Model 7]
|
v
SIGNATURE --- Falcon-512 in browser WASM (user),
|           ML-DSA-65 (custodian, oracle, admin)
|
v
OFF-CHAIN PQ VERIFICATION --- seven-condition check;
|                           failure is named, nothing downstream runs
|
v
EXECUTION --- orchestrator submits the contract call;
|           the chain re-checks its own invariants
|
v
ENVELOPE HASH = Keccak-256(envelope with signature field zeroed)
|
v
60-SECOND EPOCH --- leaves appended in arrival order
|
v
MERKLE ROOT --- one sorted-pair Keccak-256 tree per epoch [Model 8]
|
v
IMMUTABLE ON-CHAIN ANCHOR --- {root, leafCount, epoch}, ~43 k gas
|
v
OFFLINE INCLUSION PROOF --- user downloads {leaf, proof, epoch};
|                           verified against the chain alone

```

Figure 2. The authentication fabric, orthogonal to Fig. 1. The cost of the anchor does not depend on the number of events in the epoch, which is the property the whole architecture is built to exploit (§III, Model 8).

```

UNREGISTERED
| provider submits metadata, valuation basis, custodian
v
ATTESTATION RECEIVED
| ML-DSA-65 signature over the reserve statement
v
SIGNATURE CHECK ----- fail -> REJECTED (T2)
|
v
SEQUENCE CHECK ----- s <= s_prev -> REJECTED (replay)
|
v
RESERVE CHECK ----- reserve < intendedSupply -> REJECTED
|
v
VALUATION CHECK ----- age > maxStaleness -> REJECTED (stale)
|
v
LIVE <---- the only state from which minting is permitted
| reserve re-attestation overdue, or custodian key revoked
v
FROZEN --- transfers and minting stop; redemption path stays open

```

The state carried through those transitions is small and entirely on-chain:

```

Asset {
  assetId          bytes32
  assetType        enum { GOLD, TBILL, SUKUK, REIT }
  custodian        address // key resolved in PQKeyRegistry
  reserveAmount    uint256 // attested physical units
  attestedSupply   uint256 // tokens permitted to exist
  price            uint256 // last committed oracle round
  priceTimestamp   uint64
  attestationSequence uint64 // strictly increasing
  status           enum { UNREGISTERED, PENDING, LIVE, FROZEN }
}

```

Three of those fields carry semantics that the rest of the protocol depends on, so they are fixed here. `reserveAmount` is a quantity of *physical units* in the custodian’s stated denomination — grams of gold, face value of a bill — never a monetary value; valuation

is the oracle’s job and is deliberately not mixed into the reserve statement, so a price move can never make an asset look over- or under-reserved. It changes only when a new attestation with a higher sequence supersedes the previous one, which is how a custodian reports a deposit or a withdrawal of physical stock. `attestedSupply` is the token quantity permitted to exist against that reserve, and the invariant $attestedSupply \leq reserveAmount$ is enforced on every mint and re-checked on every attestation. Redemption is the mirror of issuance: burning tokens lowers `attestedSupply` first, and the custodian’s next attestation lowers `reserveAmount` to match once the physical unit has left custody — in that order, so the invariant is never violated even transiently, and a redemption in flight simply leaves the asset over-reserved.

The case that must not be left implicit is a custodian key revoked, or an attestation found false, *after* tokens are already issued. Nothing is retroactively unwound, because the tokens represent claims real investors hold. The asset moves to FROZEN: minting and transfers stop, so the false supply cannot spread, while the redemption path stays open and the collateral already locked in the vault keeps its last committed price. Positions collateralised by that asset are not liquidated by the freeze itself — the oracle heartbeat governs that — and the residual exposure, if the reserve truly does not exist, is a credit loss to be resolved off-protocol against the custodian, which is the honest boundary of what a signature can guarantee (§IV). An asset reaches LIVE only when

$$sigValid \wedge s > s_{prev} \wedge R \geq \Sigma_{intended} \wedge (t_{now} - t_{price}) \leq \tau_{max},$$

where R is the attested reserve and $\Sigma_{intended}$ the supply the provider intends to issue. The conjunction is evaluated twice, in two places that do not trust each other: the post-quantum verification service checks the signature and the envelope, and AssetRegistry re-checks sequence, reserve and freshness on-chain, because the service holds no authority (§IV). This is the reason the proposal carries no separate list of functional requirements: the admissible behaviour of the registry *is* this predicate, and §VII tests it at each named clause.

C. From live asset to financial ownership. Two standards appear here and they are not alternatives; they answer different questions, and conflating them is the most common modelling error in RWA designs.

```

LIVE ASSET
| token config: decimals, compliance modules, cap = attestedSupply
v
ERC-3643 TOKEN "who may hold a unit of this asset"
| transfer attempted -> IdentityRegistry claim check inside the token
v
VAULT (ERC-4626) "how much of the pool does a holder own"
|
v
INVESTOR HOLDS SHARES

```

ERC-3643 represents *restricted ownership*: the compliance rule lives inside the token contract, so an unauthorised transfer reverts even when it is called directly, bypassing every application server. ERC-4626 represents *pooled accounting*: it converts a quantity

of underlying units into a proportional claim, and standardises that conversion so other contracts can compose with it. The ownership abstraction the investor actually holds is therefore two levels deep,

```
physical unit -> ERC-3643 unit -> vault holds units
               -> investor holds ERC-4626 shares,
```

ERC-3643 enforces *whether* a unit may be transferred to a given identity — the check runs inside the token’s own transfer function — while ERC-4626 defines *how* the units the vault holds are converted into proportional shares. Because shares are the object a lender must value, price and seize, the correctness of the share conversion is a solvency property and not an accounting convenience — which is why it is opened first in §III.

D. Investment flow.

```
INVESTOR
| identity verified; ONCHAINID claim issued; KYC documents
| encrypted under ML-KEM-1024, only the hash on chain
v
MOCK PKR SETTLEMENT --- IFiatRail double-entry ledger, deterministic
|                        delay, injectable failure
v
TEST-NET USDC credited as the unit of account
|
v
DEPOSIT INTENT signed in the browser (Falcon-512, key never leaves device)
|
v
PQ ENVELOPE -> verification -> vault call
|
v
SHARES MINTED, rounded against the depositor
|
v
ENVELOPE HASH -> current epoch -> anchored within 60 s -> proof served
```

The settlement leg is the only part of the flow that is simulated rather than real, and it is isolated behind one interface so that replacing it with a licensed provider is a configuration change and that provider’s integration tests, not a redesign (§V). Everything downstream of the USDC credit — the cryptography, the contracts, the accounting, the risk engine and the tests — is the production article. The flow has now reached the first mechanism that requires mathematics, and the next section opens it.

III. PROTOCOL MECHANISMS AND MATHEMATICAL MODELS

Each mechanism below is reached by the flow of §II in the order it appears there, and each is carried through the same chain: why it exists, the state it operates on, its inputs, its model, the algorithm, where the algorithm runs, what it writes, who reads that, how it fails, the invariant that must survive the failure, and the measurement that tests it. Nothing is described twice; where a later model consumes an earlier one, it says so and does not restate it.

A. Model 1 — Vault accounting and the rounding boundary.

1) *Why it exists:* The investor’s claim is a share, and a share is a ratio, not a balance. Every later mechanism — collateral value, health factor, liquidation seizure — multiplies that ratio by a price. A share price that can be manipulated is therefore

not an accounting inconvenience but a solvency hole one level below the risk engine, and the manipulation is cheapest exactly at the scale this project targets: a platform whose typical deposit is PKR 1,000.

2) *State and inputs:* The vault holds A units of the underlying ERC-3643 token and has S shares outstanding. Inputs are a deposit amount a , a redemption amount s , and the externally observable balance A , which anybody can increase by transferring tokens to the vault without minting — a *donation*.

3) *The vulnerability, stated exactly:* Unmodified ERC-4626 converts by $\text{shares}(a) = \lfloor aS/A \rfloor$. An attacker mints one share with a one-unit deposit, donates X units, and A becomes $X + 1$ while $S = 1$. A later honest deposit of a units yields $\lfloor a/(X + 1) \rfloor = 0$ shares whenever $a \leq X$: the depositor receives nothing and the attacker redeems the donation plus the victim’s principal. The attack costs $X \approx a$, so at $a = \text{PKR } 1,000$ it costs PKR 1,000.

4) *Mechanism and model:* A virtual offset $\sigma = 10^6$ is added to the share supply and one unit to the assets, so the vault behaves as though it were already seeded:

$$\text{shares}(a) = \left\lfloor \frac{a(S + \sigma)}{A + 1} \right\rfloor, \quad (1)$$

$$\text{assets}(s) = \left\lfloor \frac{s(A + 1)}{S + \sigma} \right\rfloor, \quad (2)$$

with the floor on deposit and mint-quote, and the ceiling on withdrawal quote and redemption cost, so that every rounding error is resolved against the user initiating the operation and in favour of the pool. The attacker must now donate X such that $a(1 + \sigma)/(X + 1) < 1$, i.e.

$$X > a\sigma - 1 \approx 10^6 a, \quad (3)$$

which converts an attack costing one deposit into an attack costing a million deposits — the loss is bounded by σ rather than by the honesty of the first depositor.

5) Algorithm, location, writes and readers:

```
deposit(a)
|
v
convertToShares(a) -- floor, offset applied
|
v
_mint(receiver, shares) ; SafeERC20 pull of a units
|
v
writes: totalSupply, balanceOf, (A implicitly)
|
v
read by: CollateralManager (locked share balance),
        Liquidator (seizure quantity), risk engine (position value)
```

It runs in `AssetVault.sol`, an OpenZeppelin ERC-4626 subclass overriding `_decimalsOffset()` to 6 and the four conversion entry points to fix the rounding direction explicitly rather than inheriting it.

6) *Failure, invariant, test:* The failure mode is silent: a wrong rounding direction does not revert, it leaks value one wei at a time. The invariants are that `convertToAssets` is monotonically non-decreasing in the vault’s asset balance, and that no sequence of operations lets a caller redeem more than the assets they contributed plus their pro-rata share of donations and yield. Both are property-fuzzed under Foundry over 10^6 randomised call sequences of the form deposit–donate–deposit–withdraw–redeposit, and share arithmetic is additionally proved by symbolic execution in Halmos, because rounding bugs are exactly the class that random sampling misses.

B. Model 2 — Asset valuation as a decision process.

1) *Why it exists:* Everything downstream of a deposit is priced. A price is therefore not data the protocol reads; it is a decision the protocol makes, and the sequence in which the checks are applied is the security property. Applying them in a different order — median after the deviation test, for instance — would let a single corrupted feed decide whether the aggregate is accepted.

2) *Inputs and state:* Five independent price sources per asset, polled in parallel with a 3-second timeout, and the protocol’s own history: the 30-minute time-weighted average P_{TWAP} and the last committed round. A round is $\{\text{roundId}, P, t, k\}$ for k contributing sources.

3) *The pipeline, and why in this order:*

```

5 price sources (parallel, 3 s timeout)
|
v
drop invalid / stale sources    <- removes a dead feed's influence
                                before it can shift a statistic
|
v
quorum test: k >= 3 else abandon <- a statistic over two points has
                                no resistance to one bad point
|
v
P_median = median(P_1 ... P_k)  <- bounded influence: one source
                                cannot move the median at all,
                                two cannot move it past a third
|
v
deviation test against TWAP    <- catches what the median cannot:
                                a correlated majority move
|
v
monotonic roundId, timestamp    <- ordering cannot be rewritten
|
v
ACCEPTED ROUND -> signed (ML-DSA-44) -> committed on chain

```

Formally, with

$$\Delta = \frac{|P_{\text{median}} - P_{\text{TWAP}}|}{P_{\text{TWAP}}}, \quad (4)$$

the round is committed if and only if

$$k \geq 3 \wedge \Delta \leq \delta \wedge t > t_{\text{prev}} \wedge \text{roundId} > \text{roundId}_{\text{prev}}, \quad \delta = 0.10. \quad (5)$$

The median bounds the influence of a minority of corrupted feeds; the TWAP deviation gate bounds the influence of a correlated majority over a short window; the monotonic identifier makes the accepted history append-only. An attacker who moves one source cannot move the median, and one who moves the median briefly cannot move the 30-minute average.

4) *What "independent" means, and where 10% and 30 minutes come from:* Five sources are only five sources if their failures are uncorrelated, so independence is a selection criterion and not an adjective: the set is chosen so that no two feeds share a price origin, an operator or a hosting region, and at most two derive from the same underlying market. Because that is an assumption about the outside world rather than a property of the protocol, it is the one the design refuses to rely on — the quorum tolerates two dead feeds, the median tolerates two corrupted ones, and the TWAP gate is what remains when the independence assumption itself fails and three move together. Correlated majority capture is therefore not defended by the median at all; it is defended by the gate, and if the move persists past the TWAP window it is defended by the borrowing freeze rather than by a price the protocol pretends to know.

The deviation bound $\delta = 10\%$ is set from the class with the widest legitimate short-horizon move: gold has never moved 10% against its own 30-minute average in a 60-second window outside a market halt, so the gate rejects manipulation without rejecting reality. The 30-minute window is the shortest that makes the gate expensive to defeat — an attacker must hold the manipulated price across the whole window to drag the average with it, and the cost of holding a deep spot market for thirty minutes is orders of magnitude above the value of any position this protocol will carry — while remaining short enough that a genuine repricing is admitted within one grace period rather than freezing the market for a day. Both constants are per-class configurable, and Experiment C reports the false-rejection rate at each.

5) *Location, writes, readers, failure:* Feed polling and aggregation run in Node 20 workers, which sign the aggregate with ML-DSA-44; `PriceOracle.sol` verifies the signer’s role against `PQKeyRegistry`, re-checks monotonicity on-chain and stores the round. `CollateralManager` and the risk engine read it and nothing else — no component in the system ever reads a source price directly. Failure is *explicit refusal*, never extrapolation:

```

stale feed | quorum < 3 | deviation > 10% | non-monotonic round
|
v
NO NEW PRICE COMMITTED
|
v
price age exceeds heartbeat -> borrowing and new locks freeze,
liquidation eligibility freezes, repayment and withdrawal stay open

```

The asymmetry is deliberate: a frozen oracle must not be able to create liquidations, but must never trap a user’s exit.

6) *Invariant and test:* Committed rounds are strictly increasing in identifier and timestamp, and no committed price differs from the concurrent TWAP by more than δ . Experiment C (§VII) injects one, two and three malicious sources, a stale source and a large deviation, and asserts which of the four named checks rejects each case — a rejection at the wrong check is a failure even when the overall outcome is correct.

C. Model 3 — Collateral, capacity and the health factor.

Table 1. Risk parameters per asset class. The heartbeat is the maximum price age before borrowing freezes; it tracks how fast the asset can move, not how fast the feed can poll.

Asset class	θ	λ	$\beta_{\max}$	heartbeat
Gold (mandatory)	60%	75%	5%	60 s
Treasury bill	75%	85%	3%	300 s
Sukuk	65%	80%	4%	300 s
Fractional REIT	40%	60%	10%	3600 s

1) *Why it exists*: This is the mechanism that turns an owned share into borrowing power, and it is the only place where the protocol’s solvency is defined. It runs on locked shares and committed prices — the outputs of Models 1 and 2 — and nothing else.

2) *Model*: With q_i the quantity of collateral asset i locked by the position, p_i its last committed price, θ_i its maximum loan-to-value, $\lambda_i > \theta_i$ its liquidation threshold and D the position’s debt:

$$C = \sum_i q_i p_i, \quad \text{LTV} = \frac{D}{C}, \quad (6)$$

$$\text{BorrowCapacity} = \sum_i q_i p_i \theta_i - D, \quad (7)$$

$$\text{HF} = \frac{\sum_i q_i p_i \lambda_i}{D}. \quad (8)$$

The protocol supports multiple collateral assets in one position, so BorrowCapacity is written as the weighted sum throughout and is never abbreviated against the unweighted total C ; in the single-collateral case it reduces to $C\theta - D$, and that is the only form in which the shorter notation is used.

The gap $\lambda_i - \theta_i$ is the design freedom: it is the price movement a position can absorb between origination and eligibility for liquidation, and it is set per asset class from that asset’s observed volatility and its settlement latency (Table 1).

3) *Where the numbers come from*: Every value in Table 1 and in the band structure below is derived from one of three quantities and is re-derived, not inherited, whenever a parameter changes.

Loan-to-value θ_i is set so that the collateral covers the debt through a one-day adverse move at the class’s historical 99th percentile plus the cost of liquidating it. Gold’s daily volatility is roughly 1% with fat tails around monetary events, and its price is continuous and deeply liquid, so $\theta = 60\%$ leaves a 40% cushion — an order of magnitude more than the worst observed single-day move. Treasury bills carry duration risk only and mature to par, so the same reasoning admits $\theta = 75\%$; Sukuk sit between the two because their secondary market is thinner; fractional REITs are marked from infrequent appraisals rather than trades, so $\theta = 40\%$ prices the staleness of the mark and not the volatility of the asset.

Liquidation threshold λ_i is then set so that $\lambda_i - \theta_i$ exceeds the largest price move that can occur *inside one heartbeat* at that class’s volatility, with margin. Gold’s 15-point gap at a 60-second heartbeat is roughly fifteen times its per-minute standard

deviation; the REIT’s 20-point gap at a one-hour heartbeat covers a full day of appraisal drift. The heartbeat is therefore not a polling convenience — it is the term over which $\lambda_i - \theta_i$ must hold, which is why the two are set together and tested together.

Band boundaries follow from the same reasoning applied to time rather than price. The 1.15 origination bound is θ/λ at gold’s parameters plus a margin, so a freshly opened position sits at least one full heartbeat’s worth of volatility away from the warning band. The 0.95 liquidation point is the depth at which a breach can no longer be assumed transient: at gold’s per-minute volatility, recovering 5% inside the 6-hour grace window is a regime change rather than a wick, so below it the buffer’s premise fails and the ordinary path must run. The 50% close factor is chosen because it is the smallest fraction that restores $\text{HF} > 1$ from the deepest breach the buffer can hand over — shown in Model 6 — so no larger seizure is ever necessary in the ordinary case.

Every one of these numbers is an input to the evaluation, not an assumption of it: Experiment B sweeps θ , λ and the band boundaries across the historical and Monte Carlo paths, and a parameter that produces bad debt at 3σ is reported as such.

4) *The state machine the value drives*:

HF >= 1.15	borrowing permitted; new debt admitted only if the POST-state HF also satisfies this bound
v	
1.00 <= HF < 1.15	new borrowing frozen; borrower notified; 6 h top-up window opened; position otherwise normal
v	
0.95 <= HF < 1.00	reserve buffer band -----> Model 5
v	
HF < 0.95	liquidation eligible -----> Model 6
v	
HF < 0.90	full closure; residual bad debt socialised

The 1.15 origination bound is not the same number as the 1.00 solvency point by design: no position can be opened one price tick away from liquidation, which is the failure mode that makes small positions uneconomic to hold.

5) *Location, writes, readers, failure, test*:

```
CollateralManager.getCollateralValue() -- reads PriceOracle rounds
|                                     and locked share balances
v
calculateHF() -- pure function of (q, p, lambda, D)
|
v
LendingPool (admits or refuses borrow; writes scaled debt)
|
v
off-chain risk engine (Redis Streams workers) re-computes the same
function on every accepted price tick and emits notifications and
liquidation proposals -- it proposes, the chain decides
```

The engine holds no authority: a liquidation transaction re-derives HF on-chain and reverts if the engine was wrong or stale, so an engine outage delays liquidations but cannot cause an incorrect one. The invariant is that no borrow or unlock transaction may leave a post-state $\text{HF} < 1.15$, and that locked shares never exceed held shares; both are Foundry invariants, and HF

monotonicity in D and in each p_i is proved symbolically in Hal-mos. Sweep latency at 10,000 open positions is measured in Experiment E.

D. Model 4 — Interest as a global index.

1) *Why it exists:* Two separate problems are solved here, and they are usually conflated. The first is *pricing*: the rate must clear the market between borrowers and suppliers. The second is *cost*: interest must accrue continuously to every position without the protocol touching every position.

2) *Rate model:* With D total debt and L idle liquidity, utilisation is $U = D/(D+L)$, and the rate is a two-slope curve with a kink at the optimum $U^* = 0.80$:

$$r(U) = \begin{cases} r_0 + \frac{U}{U^*} s_1, & U \leq U^*, \\ r_0 + s_1 + \frac{U - U^*}{1 - U^*} s_2, & U > U^*, \end{cases} \quad (9)$$

with $r_0 = 0$, $s_1 = 4\%$ and $s_2 = 75\%$. Below the kink the rate is nearly flat, so ordinary borrowing is cheap; above it the second slope makes the cost of the last 20% of liquidity rise steeply, which is what refills the pool before withdrawals can fail. The kink is a liquidity-availability mechanism expressed as a price.

3) *Index model, and why it is the engineering point:* Accrual is not applied per borrower. A single global borrow index compounds:

$$I_{t+\Delta t} = I_t \left(1 + r(U) \frac{\Delta t}{T_{\text{year}}} \right), \quad (10)$$

and each borrower stores a *scaled* balance b_u fixed at the moment of borrowing, with the true debt resolved only when read or modified:

$$D_u(t) = b_u I_t, \quad b_u = \frac{D_u(t_0)}{I_{t_0}}. \quad (11)$$

naive:	every rate change -> loop over every borrower	$O(n)$ per update
index:	every rate change -> one index write	$O(1)$ per update
	debt resolved at read time	$O(1)$ per query.

This converts interest accrual from an $O(n)$ per-update operation into an $O(1)$ global index update, with per-user debt resolved only on query or modification. The consequence is the reason the design is viable at all at micro-position scale: 10,000 open positions cost exactly as much to accrue as one, so the protocol's gas cost does not scale with the number of small borrowers it serves.

4) *Location, failure, invariant, test:* LendingPool.sol holds I , $\sum b_u$ and U ; the index is updated lazily at the start of every state-changing call, so an idle protocol costs nothing. The failure mode is precision: repeated multiplication of a ray-scaled index accumulates error, and a downward error is theft from suppliers. The invariant is that total debt equals the sum of scaled balances times the index to within one wei per position, and that I is non-decreasing. It is fuzzed over randomised time skips and rate paths, including the $U \rightarrow 1$ boundary.

E. Model 5 — The bounded reserve buffer.

1) *Why it exists:* A liquidation is a permanent transfer of value away from a borrower, priced for positions where the bonus is small relative to the position. At PKR 20,000 a 5% bonus is PKR 1,000 — more than a year of expected appreciation on the collateral — and a single transient wick below HF = 1 that recovers minutes later costs the borrower that amount permanently. The buffer introduces a state between healthy and liquidated, in which a *shallow* and *brief* breach is carried rather than realised.

2) *State and model:* The buffer holds B in the settlement asset, funded from a share of protocol interest. When a position enters the band $0.95 \leq \text{HF} < 1.00$, its shortfall is the debt the collateral no longer covers at the liquidation threshold:

$$\text{shortfall} = D - \sum_i q_i p_i \lambda_i = D(1 - \text{HF}). \quad (12)$$

The buffer issues a *claim* for that shortfall rather than paying it. The claim is an accounting reservation, not a transfer of settlement assets: the position stays open, the borrower's debt remains D and continues to accrue interest at the index of Model 4, no collateral moves, and the reserved amount is counted against the buffer's remaining capacity for the epoch so it cannot be promised twice. What the buffer absorbs during the grace period is therefore not the borrower's debt but the protocol's *exposure* — the liquidation that solvency would otherwise require is deferred against a reservation that is either released at no cost or paid when the timer expires. Nothing is forgiven: if the claim is realised the buffer pays the shortfall into the pool so that suppliers are made whole, and the position proceeds to liquidation with its debt undiminished. The claim is subject to a per-epoch absorption cap:

$$\sum_{\text{claims in epoch } e} \text{shortfall} \leq \kappa B_e, \quad \kappa = 0.40, \quad (13)$$

with the buffer evolving as

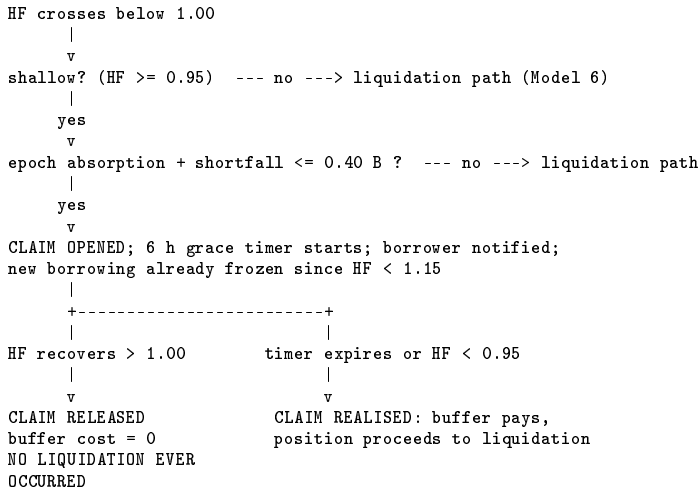
$$B_{e+1} = B_e + f_e - \alpha_e, \quad (14)$$

where f_e is protocol funding in the epoch and α_e is the shortfall actually absorbed — realised only for claims that expire, since a claim released on recovery costs the buffer nothing.

3) *Why the cap exists:* Not "to protect reserves". Without the cap, a market-wide move lets the positions that breach *first* consume the entire buffer, leaving it empty exactly when the later and deeper breaches arrive; the mechanism would convert itself from a borrower protection into a source of systemic bad debt, and its benefit would accrue by arrival order rather than by need. The cap converts the buffer from an unbounded loss absorber into a bounded first-loss mechanism whose maximum per-epoch exposure is known in advance and is independent of how many positions breach.

4) *Who funds it, who owns it, and what happens at scale:* The buffer is funded from a fixed share of protocol interest revenue — the spread between what borrowers pay and what suppliers receive — so it is capitalised by the borrowing activity whose risk it insures, and no depositor’s principal is ever at risk in it. It is protocol-owned and not depositor-owned: it holds no claim on any vault’s assets and confers no claim on any user, which is what allows it to pay into the pool without renegotiating any position. Under mass stress its behaviour is defined entirely by the cap, and this is the case worth stating explicitly: if a thousand positions breach in one epoch, claims are opened in arrival order until the cumulative reserved shortfall reaches κB_e , and every position after that point takes the ordinary liquidation path immediately. The mechanism therefore has no queue, no pro-rata rationing and no dependence on the order in which the risk engine happens to sweep — it has a hard ceiling per epoch and a well-defined behaviour above it. If the buffer is already exhausted when an epoch opens, the cap binds at zero and the protocol behaves exactly as the control build does.

5) Lifecycle:



6) *Location, readers, failure, invariant:* `ReserveBuffer.sol` holds B , the per-epoch absorbed total and the open claims; Liquidator must consult it before any seizure, so the band is enforced on-chain and not by the off-chain engine. If the buffer is empty or the cap is already consumed, the mechanism degrades exactly to the control behaviour — immediate liquidation — which is the correct degradation, because the buffer is an improvement over the baseline and never a dependency of it. The invariants are that absorption per epoch never exceeds κB , that a released claim returns the full reserved amount, and that no position can hold two simultaneous claims.

7) *Test:* This is the mechanism’s whole justification, so it is measured rather than asserted. Experiment B replays the 2020 and 2024 gold drawdowns and 10,000 Monte Carlo GBM paths at 1σ , 2σ and 3σ against an Aave-V3-parameterised control, reporting liquidations avoided, bad debt, peak buffer utilisation

and value returned to borrowers. The hypothesis is falsifiable in both directions: the buffer must reduce liquidations caused by transient shocks *and* must not allow sustained undercollateralisation to persist, and the 3σ paths exist specifically to find the point at which the cap binds before the buffer empties.

F. Model 6 — Liquidation with a decaying bonus.

1) *Why it exists:* When a breach is neither shallow nor transient, the protocol must restore solvency, and it must pay somebody to do it. The design question is not whether to pay but how the payment behaves in time.

2) *Model:* Eligibility requires $HF < 0.95$ and no live buffer claim. A liquidator repays x of the debt, bounded by the close factor $\phi = 0.50$:

$$x \leq \phi D, \quad (15)$$

and seizes collateral worth $x(1 + \beta(t))$ at the committed price,

$$q_{\text{seized}} = \frac{x(1 + \beta(t))}{p_i}, \quad (16)$$

where the bonus decays linearly over $N = 20$ blocks from the moment of eligibility at block t_0 , from the per-class $\beta_{\max}$ of Table 1 down to a floor $\beta_{\min} = 0.6\beta_{\max}$ (3% for gold):

$$\beta(t) = \beta_{\max} - (\beta_{\max} - \beta_{\min}) \min\left(1, \frac{t - t_0}{N}\right). \quad (17)$$

Half the position is closed rather than all of it, so a borrower who breaches by a small margin is not fully unwound to cure it; and because the seizure is priced at a committed round, a liquidator cannot profit from a price the protocol has not accepted.

3) *Why the bonus decays:* A fixed bonus is a fixed prize awarded to whoever wins a race in the first block after the breach, which is the definition of an MEV auction: the value is competed away into priority fees and the borrower pays the maximum regardless. A bonus that starts at $\beta_{\max}$ and decays toward $\beta_{\min}$ over 20 blocks changes the incentive from racing for the largest reward to accepting progressively smaller compensation for a delayed liquidation. The first block still carries the strongest incentive, so a deeply unsafe position is cleared immediately; but a liquidator who is willing to act at less than $\beta_{\max}$ has a strictly better strategy than out-bidding the field in block one, and every block of delay that the position survives — including a recovery that removes the eligibility altogether — costs the borrower less. The mechanism buys the borrower the cheapest liquidation that anyone is willing to perform rather than the most expensive one that anyone is willing to race for.

The incentive at block 20 is $\beta_{\min}$, and it does not fall further: below that floor no liquidator would cover gas and price risk, and an uncleared unsafe position is worse for every supplier than a cheaply cleared one. If a position is still unliquidated at the floor — the case where the collateral is too illiquid to be worth $\beta_{\min}$ to anyone — it crosses $HF < 0.90$ and takes the full-closure path, where the residual is socialised rather than left open.

4) *Does a 50% close actually restore solvency?:* It must be shown rather than assumed. Repaying $x = \phi D$ and seizing $x(1 + \phi)$ of collateral leaves

$$HF' = \frac{\sum_i q_i p_i \lambda_i - \phi D(1 + \beta) \bar{\lambda}}{(1 - \phi)D}, \quad (18)$$

where $\bar{\lambda}$ is the threshold of the seized asset. For a single-collateral position this is $HF' = (HF - \phi(1 + \beta)\bar{\lambda})/(1 - \phi)$, so with $\phi = 0.5$, $\bar{\lambda} = 0.75$ and $\beta = 0.05$ a position handed over at the buffer's floor, $HF = 0.95$, returns to $HF' \approx 1.11$ — above the solvency point, below the origination bound, which is the intended landing state: solvent, but not permitted to add new debt until the borrower tops up. The close factor is set at the smallest value for which this holds across every class in Table 1 at the deepest breach the buffer can hand over.

If it does *not* hold — a gap move that carries the position past $HF < 0.90$ before any liquidator acts — a second liquidation is permitted in the same block, and below 0.90 the close factor is lifted to full closure with the residual socialised from the buffer first and from supplier shares thereafter. The invariant tested is not "a liquidation restores $HF > 1$ " but the weaker and actually true "every liquidation strictly increases HF ", which is what makes repeated application terminate.

5) *Location, writes, readers, failure, invariant, test:* Liquidator.sol executes; it reads CollateralManager for HF, ReserveBuffer for a live claim, PriceOracle for the committed round, and writes debt reduction and share transfer through LendingPool and the vault. Failure of the off-chain engine delays detection but cannot cause an unjustified seizure, because every clause is re-checked on-chain. The invariants are that a liquidation cannot execute while a claim is live, that $x \leq \phi D$, that post-state HF is strictly greater than pre-state HF, and that seizure uses a price no older than the asset's heartbeat. Front-running behaviour is measured in Experiment B alongside the buffer, since the two mechanisms interact on exactly the same positions.

G. Model 7 — Post-quantum authentication and the envelope.

1) *Why it exists:* A tokenised asset's record states who owns what fraction, backed by what reserve, attested by whom and when, and it must remain checkable for as long as the asset exists — commonly thirty years. Its authenticity today rests entirely on elliptic-curve signatures, whose hardness assumption has a shorter expected life than the record. The attack requires no quantum computer now, only storage: an adversary archives signed authorisations today and forges retroactive ownership once ECDLP falls.

2) *One algorithm per job — the complete assignment:* A single primitive everywhere would be simpler and worse, because the jobs have opposite cost functions: a signature the chain must hold is priced by its bytes, a signature produced on a server is priced by its side-channel risk, and the key that can revoke all

Table 2. Primitive assignment. Every post-quantum operation in the system is one row of this table.

Actor	Primitive	Why this one
User device	Falcon-512	the most compact PQ signature (≈ 666 B), which matters because every user action carries one; signing happens in the browser where the key never leaves the device
Custodian	ML-DSA-65	attestations are the highest-value assertions in the system; category-3 parameters and deterministic, integer-only signing, so no floating-point side channel on a service host
Oracle	ML-DSA-44	price rounds are frequent and short-lived; category-2 is sufficient for an assertion superseded within a heartbeat, and the smaller parameter set lowers per-round overhead
Administration	ML-DSA-65	parameter changes are rare, durable and high-impact, so they take the stronger service parameter set
Root of trust	SLH-DSA (SHA2-128s)	hash-based: the authority to revoke and rebind every lattice key must not itself rest on lattice hardness; slow and large, which is acceptable for an offline key used a few times a year
Confidentiality	ML-KEM-1024 (at rest), hybrid X25519 + ML-KEM-768 (in transit)	documents need thirty-year secrecy against harvest-now-decrypt-later; the transit hybrid hedges implementation risk in a young standard

others is priced by the diversity of its assumption. Table 2 is the whole assignment, stated once; no later section re-argues an algorithm choice, and any actor not in this table has no authority in the protocol.

One asymmetry in the table is deliberate and easy to miss: Falcon is *verified* everywhere but *signed* only on user devices. Its Gaussian sampling over floating-point lattices is a documented side-channel and correctness hazard, which is a risk a browser tab holding one user's key can carry and a shared service host holding the protocol's keys cannot.

3) *The envelope:* No signature is ever taken over a raw payload. Every post-quantum signature in the system covers one canonical structure whose fields kill one attack each, so replay resistance is a property of the format rather than of careful coding in each handler (Table 3).

4) *The seven-condition check:* One verifier routine, shared by client and every service, accepts an envelope only if all seven hold: (1) the algorithm is allow-listed for that action type; (2) the signer resolves to an active, unrevoked key inside its validity window; (3) the domain separator equals the locally recomputed value; (4) the clock lies inside the validity window; (5) the nonce is exactly the stored value plus one; (6) the signature verifies over the serialised envelope with the signature field zeroed; (7) the payload hash matches the payload being executed. A failure is rejected *at a named check*, and §VII asserts the name, not

Table 3. Envelope fields and the specific attack each one removes. The leaf that is later anchored is the hash of the whole structure, so the anchor commits to the signature and to its context.

Field	Security function
version	format evolution without ambiguity in the signed bytes
algorithm	cryptographic agility; prevents downgrade to a retired primitive
chainId	cross-chain replay protection
contract	cross-contract replay protection
action	action-substitution protection: an intent is meaningless for any other action
signer	binds the envelope to a key record in PQKeyRegistry
nonce	same-signer replay protection; strictly incrementing per signer and action
validFrom, validTo	delayed-execution protection; 90 s for user intents, 3600 s for attestations
payloadHash	payload-substitution protection: the executed payload cannot differ from the signed one

merely the rejection.

5) *Location, writes, readers, failure*: Signing happens in the browser in Rust compiled to WASM; the private key is generated on the device, stored in IndexedDB and never transmitted. Verification is a stateless NestJS service holding no authority; it writes the envelope hash to the current epoch’s leaf list and nothing else. If the service stalls, new authenticated actions stop while existing on-chain positions remain operable directly against the contracts — the fabric can withhold new authorisations but can never move funds, because no off-chain component in the system can (§IV).

6) *Invariant and test*: No two accepted envelopes share a (*signer, action, nonce*) triple, and no envelope is accepted outside its window or under a revoked key. The adversarial suite replays captured envelopes, reuses nonces, confuses domains across contracts and chains, and submits expired intents; NIST CAVP vectors are run for all six primitives in both native and WASM builds, so conformance is tested rather than asserted. Signing and verification latency and signature size are Experiment A’s RQ1 measurements.

H. Model 8 — Merkle anchoring and the cost argument.

1) *Why it exists*: Model 7 gives post-quantum authentication off-chain, but off-chain verification alone leaves no durable evidence: a platform that verifies its own signatures can also rewrite its own history. The question is how to obtain permanent, publicly checkable evidence without paying to verify a post-quantum signature on-chain — which Table 4 shows is not affordable today.

2) *Model*: Each verified envelope contributes a leaf

$$\ell_i = \text{Keccak256}(\text{envelope}_i), \quad (19)$$

Table 4. On-chain cost of authentication options, and the verdict each one earns. The last row is this design.

Mechanism	Gas	Verdict
ecrecover (ECDSA)	3,000	cheap, not quantum-safe — the baseline to beat
Falcon-512 verifier in pure Solidity	≈1,500,000	≈500× ECDSA; retained for the dispute path only
ML-DSA via EIP-8051 precompile	4,500 + 20.5 kB calldata	draft; calldata, not compute, dominates cost
Falcon via EIP-8052	≈3,000 projected	would be ideal; not shipped, so unusable today
Merkle inclusion proof, depth 12	≈22,000	paid only when a proof is checked on-chain
Epoch anchor — this design	≈43,000 per epoch, $O(1)$ in events	≈430 gas/event at $n = 100$; below ECDSA past $n \approx 14$

appended in arrival order. Every 60-second epoch e the batcher builds one sorted-pair Keccak-256 tree and writes a single record:

$$R_e = \text{Merkle}(\ell_1, \dots, \ell_n), \quad \text{anchor} = \{R_e, n, e\}. \quad (20)$$

Inclusion of ℓ_i is proved by $\lceil \log_2 n \rceil$ sibling hashes against R_e , checkable on any machine that has never contacted the platform.

3) *The break-even derivation*: Let $G_{ec} = 3,000$ be the per-event cost of the ECDSA baseline, $G_{fal} \approx 1.5 \times 10^6$ the cost of on-chain Falcon verification, $G_a \approx 43,000$ the anchor write, $G_p \approx 22,000$ a depth-12 on-chain proof check, and ρ the fraction of events whose proof is ever disputed on-chain. Then per epoch of n events:

$$\text{Cost}_{\text{direct}}(n) = n G_{fal}, \quad (21)$$

$$\text{Cost}_{\text{anchor}}(n) = G_a + \rho n (G_p + G_{fal}), \quad (22)$$

and the marginal cost per authenticated event is

$$g(n) = \frac{G_a}{n} + \rho (G_p + G_{fal}). \quad (23)$$

With $\rho = 0$ — the undisputed case, which is the overwhelming majority — anchoring is cheaper than the ECDSA baseline exactly when

$$\frac{G_a}{n} < G_{ec} \iff n > \frac{43,000}{3,000} \approx 14.3, \quad (24)$$

so from roughly fifteen authenticated events per 60-second epoch onward, the post-quantum system costs *less* on-chain gas per authenticated action than the non-post-quantum system it replaces, while providing evidence the baseline cannot. Against direct on-chain post-quantum verification the comparison is not close at any n . The dispute path is where ρ enters: one contested envelope is settled by the 1.5 M-gas Falcon verifier, paid once and by the challenger, so the expensive verifier is retained as a court and not as a toll booth.

4) *Where 60 seconds, 43 k gas and the sorted-pair tree come from:* The epoch length is the only user-visible latency the mechanism adds, and it is bounded on both sides. Shorter epochs anchor more often, so the fixed G_a is amortised over fewer events and the break-even n^* is reached less often; longer epochs leave a wider window in which a user has executed but cannot yet be given a proof. Sixty seconds is chosen to match the liquidation heartbeat of the fastest asset class, so the anchoring cadence never becomes the slowest clock in the system, and it puts proof availability inside the same order of magnitude as block confirmation, which is what RQ2 measures.

The ≈ 43 k figure is a budget, not a measurement: one cold SSTORE of the root (20 k), a second slot for leaf count and epoch, the transaction's 21 k base cost and the event log, with the remainder as margin. Experiment A replaces it with measured Foundry snapshots, and the break-even moves with it — which is why n^* is reported as an outcome rather than quoted as a design constant.

Sorted-pair hashing — ordering each sibling pair before hashing — is used because it removes the need to transmit the position bits of a proof and, more importantly, because it is the construction OpenZeppelin's audited MerkleProof and every independent verifier already implement. An audit trail whose proofs can only be checked by our own code is not an audit trail. The leaves are the hashes of full envelopes, so second-preimage confusion between a leaf and an internal node is prevented by the fixed leaf domain rather than by the tree shape.

5) *What the anchor proves, and what it does not:* It proves that a specific envelope — with its signer, action, nonce, validity window and payload hash — existed and was accepted by the platform no later than the block in which its epoch root was written, and that the platform cannot later deny, alter or reorder it. Three things it does *not* prove, stated here so no later section implies otherwise: it does not prove the envelope's *signature* was valid, since that check happened off-chain and is only re-derivable by rerunning verification against the key registry (the dispute path); it does not prove the platform *executed* the envelope, which is proved separately by the contract state the transaction wrote; and it does not prove *completeness* — a platform that silently drops an envelope before it becomes a leaf produces no evidence of the omission, which is why the user's client retains its own signed copy and can present it against a root that does not contain it.

6) *The six steps, and what each guarantees:*

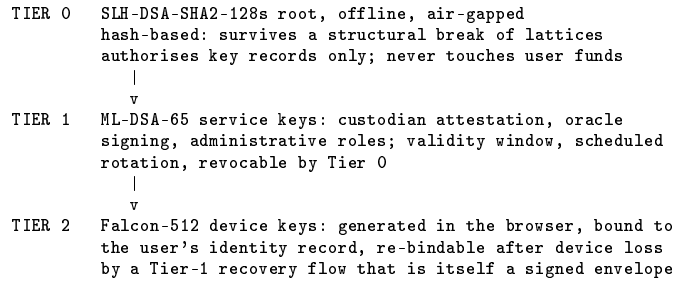
- | | |
|------------|--|
| 1. SIGN | client builds the envelope and signs in WASM; the key never leaves the device |
| 2. VERIFY | the seven-condition check; failure is named and nothing downstream runs |
| 3. EXECUTE | the orchestrator submits the contract call; the chain enforces its own invariants independently |
| 4. LEAF | the envelope hash is appended to the current epoch |
| 5. ANCHOR | one root written per 60 s, ~ 43 k gas, independent of n |
| 6. PROVE | the user downloads {leaf, proof, epoch} and verifies it offline against the chain, trusting nothing else |

7) *Location, failure, invariant, test:* The batcher is a Redis-Streams consumer; PQAnchor.sol is immutable and non-upgradeable, because an upgradeable anchor is not an anchor. If the batcher fails, execution is unaffected — the chain is still the system of record — and the commitment is merely delayed: the next anchor carries the accumulated leaves and the epoch numbering makes the gap explicit rather than hiding it. Invariants: epoch numbers strictly increase, a written root is never overwritten, and the stored leaf count equals the number of leaves in the tree. Experiment A measures the amortisation curve from $n = 1$ to $n = 4096$ and locates the empirical break-even against ecrecover on real Foundry gas snapshots rather than the arithmetic above.

1. *Model 9 — The key hierarchy.*

1) *Why it exists:* Every mechanism above ends at a signature, and a signature is only as meaningful as the answer to "which key, and was it valid then?" A compromise at any level must be recoverable by something above it, and the top level must not share an assumption with the levels below — otherwise a structural break in lattice cryptography takes the recovery mechanism down together with the thing it was meant to recover.

2) *Structure:*



3) *Lifecycle and the historical-validity property:* Registration binds a public-key hash, an algorithm identifier, a role and a window [validFrom, validTo] in PQKeyRegistry. Rotation registers the successor before retiring the predecessor, so no gap exists in which no key is authoritative. Revocation is never destructive: it sets validTo to the revocation instant, so future signatures are rejected while every historical signature inside the closed window remains verifiable. That property is only meaningful because Model 8 provides an independent time reference — the anchored epoch proves the envelope existed at a specific block, so "signed before revocation" is a checkable claim rather than a stored assertion. A thirty-year audit requirement cannot be met by a registry that deletes keys.

4) *Agility:* Each algorithm identifier is a byte in the envelope and an entry in an on-chain allow-list, not a compile-time constant. Retiring a primitive is therefore a governance transaction that sets validTo on the affected keys; historical signatures stay verifiable, and when a Falcon precompile eventually ships, moving to direct on-chain verification is a change of verifier

address rather than a migration. This is the property that makes a thirty-year record plausible: the system is designed to outlive its own primitives.

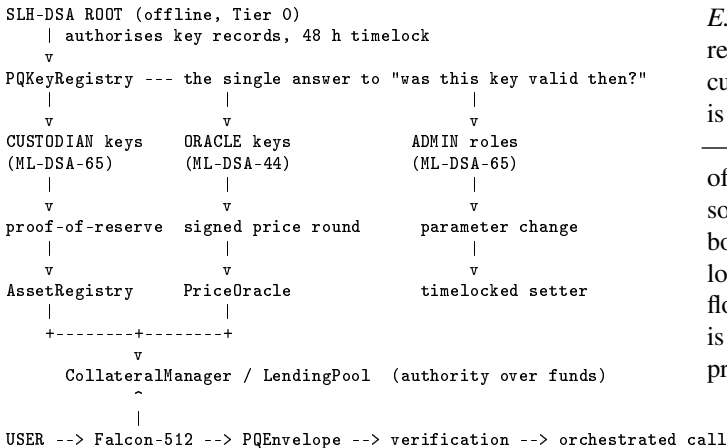
5) *Failure, invariant, test*: The catastrophic case is Tier-0 compromise, which is why it is offline, split across roles, and gated by a 48-hour timelock on parameter changes — pausing is immediate, but repayment and withdrawal are never pausable, so a compromised operator cannot trap users. Invariants: a revoked key cannot authenticate a new action; a signature anchored before revocation still verifies; role separation is enforced on-chain so an oracle key cannot sign an attestation. Rotation and revocation are exercised end-to-end in the M2 acceptance test, with the replay suite asserting rejection at the named check.

IV. CRYPTOGRAPHIC AND TRUST ARCHITECTURE

The mechanisms are now defined, so this section answers only what remains: *who is permitted to assert what, and how does the system know?* It states the one rule that decides where logic lives, the six places where an assertion crosses from one party to another, and the consequence of both.

A. *The dividing rule*. The system divides along one line, and every earlier decision follows from it: *the chain holds authority, everything else holds work*. On-chain code is the sole system of record for ownership, debt and commitments. Off-chain services perform the expensive computation — post-quantum verification, price aggregation, risk sweeps, proof serving — and are never trusted, because whatever they assert must appear inside a signed envelope that the chain has anchored. Logic goes on-chain when it must be censorship-resistant, publicly auditable or authoritative over funds; it stays off-chain when it is compute-heavy and its result can be *committed* rather than recomputed. One consequence is worth stating plainly and is tested as such: no off-chain component can move a user’s funds — it can only prove that the user asked it to.

B. *The trust graph*. Three chains of assertion exist, and they meet only on-chain.



Every arrow into the authoritative layer passes through

Table 5. The six trust boundaries. B1–B6 are the only places in the system where an assertion changes hands.

Boundary	Mechanism	Replay defence
B1 device → platform	Falcon-512 envelope	monotonic nonce, 90 s TTL
B2 custodian → platform	ML-DSA-65 attestation	strictly increasing per-asset sequence
B3 sources → platform	TLS per source, then ML-DSA-44 over the aggregate	monotonic round identifiers
B4 platform → chain	ECDSA transaction under a role whose key record is bound by the Tier-0 root	chain nonce; on-chain re-check of every clause
B5 chain → user	Merkle inclusion proof against an anchored root	immutable anchor, epoch numbering
B6 service ↔ service	hybrid X25519 + ML-KEM-768 mTLS	session-bound

PQKeyRegistry, and no arrow bypasses it. That is the entire trust model: authority is a key record with a window, and a claim is a signature inside an anchored envelope.

C. *The six boundaries*. Each boundary has exactly one authentication mechanism and one replay defence; no boundary relies on application logic alone (Table 5).

B4 deserves one remark, because it is the only ECDSA signature that survives in the design. It authenticates a *transaction submission*, not a record: a forged submission can at worst spend the operator’s gas, since the contracts re-check the post-quantum evidence and their own invariants independently. The durable records — attestations, intents, price rounds, anchors — are all post-quantum signed, which is precisely the separation the proposal claims.

D. *Confidentiality*. Signatures protect integrity; KYC documents need secrecy, and secrecy has the same thirty-year horizon. Documents are encrypted under ML-KEM-1024 wrapping AES-256-GCM at rest, with only the content hash on-chain, and transported over hybrid X25519 + ML-KEM-768. The hybrid choice is deliberate on both sides: pure X25519 is harvest-now-decrypt-later exposure, and pure ML-KEM is unhedged against implementation flaws in a young standard.

E. *What this architecture does not defend*. Three exposures remain and are stated rather than argued away. A dishonest custodian who signs a truthful-looking but false reserve statement is detected by re-attestation and reserve invariants, not prevented — the protocol binds *who said it and when*, not the physical truth of the claim. Collusion among three or more of the five price sources defeats the median, which is why the TWAP gate and the borrowing freeze exist as second and third lines. And a user who loses the device holding a Tier-2 key requires a Tier-1 recovery flow; that flow is itself a signed, anchored envelope, so recovery is auditable, but it is an operational trust in the platform’s identity process that cryptography does not remove.

Table 6. Every mechanism of §III mapped to its implementation, the existing component it stands on, and the part that is our work.

Protocol mechanism	Implementation	Existing component	Our work
PQ signing/verification (M7)	Rust + liboqs → WASM; native for services	liboqs 0.12, PQCclean, NIST CAVP vectors	no new cryptography; conformance harness across native and WASM builds
PQ envelope (M7)	TypeScript codec + Rust codec, byte-identical	none	canonical byte layout, seven-condition verifier, nonce store
Merkle anchoring (M8)	Solidity + Redis-Streams batcher	OpenZeppelin <code>MerkleProof</code> (sorted-pair)	epoch anchor contract, proof service, independent offline verifier CLI
Key hierarchy (M9)	Solidity <code>PQKeyRegistry</code>	none	three-tier PQ registry with windows, rotation, non-destructive revocation
Asset registry (§B)	Solidity 0.8.26	OpenZeppelin access control	RWA state machine and reserve/sequence/freshness invariants
Restricted ownership (§C)	ERC-3643	T-REX, ONCHAINID	integration with the registry; claim topics for this asset set
Vault accounting (M1)	ERC-4626	OpenZeppelin 5.x	virtual offset, explicit rounding direction, RWA transfer restrictions
Oracle (M2)	Solidity + Node 20 workers	Chainlink/Uniswap aggregation and TWAP patterns	five-source pipeline with signed rounds and ordered rejection checks
Lending, interest, liquidation (M3, M4, M6)	Solidity	Aave V3 design (scaled debt, index, health factor)	RWA collateral, micro-position parameters, decaying bonus
Reserve buffer (M5)	Solidity <code>ReserveBuffer</code>	none	bounded first-loss band, epoch cap, claim lifecycle
Risk engine (M3)	TypeScript, Redis Streams consumer groups	none	sweep, notification and liquidation proposal at 10,000 positions
Settlement rail (mock)	NestJS <code>IFiatRail</code>	none	double-entry ledger, injectable failures and delays

V. PROTOCOL IMPLEMENTATION

Everything above is a mechanism. This section is how it gets built: what is reused, what is adapted, what is new, on what platform, in what order, and what closes each piece of work.

A. Mechanism to implementation. Table 6 is the honesty table of the project. It separates the three categories that a proposal of this kind most often blurs — existing components used as published, existing designs adapted, and work that does not exist elsewhere.

B. Standing on existing open-source systems. For each external system the project answers three questions — what is studied, what is reused unmodified, and what is deliberately changed. This is implementation research, not a reading list.

1) ERC-3643 / T-REX: *Study:* identity claims, transfer-restriction enforcement points, compliance module composition. *Reuse:* the standard interface and its enforcement point inside the token, because compliance that lives in the application is bypassable by calling the contract directly (this is exactly the property FR-level "compliance" claims usually fail to deliver). *Modify:* binding to `AssetRegistry`, so a token cannot exist for an asset that is not `LIVE`. *Constraint:* the T-REX reference implementation is GPL-3.0; the prototype uses it as published, and a clean-room implementation of the same interface is the production path if a permissive licence is required — which is affordable because contracts depend on the interface, not on the implementation.

2) ERC-4626 and OpenZeppelin: *Study:* asset/share conversion, rounding, and the documented inflation attack. *Reuse:* the

interface and accounting model, and the audited OpenZeppelin base. *Modify:* the virtual offset and the four conversion overrides of Model 1.

3) Aave V3: *Study:* scaled debt balances, the interest index, health-factor computation, and the liquidation call path. *Reuse:* the mathematical mechanism — this is a well-tested design and reinventing it would add risk without adding contribution. *Modify:* RWA collateral with per-class heartbeats, parameters set for PKR-scale positions, the buffer band inserted between healthy and liquidatable, and the decaying bonus. Aave V3 is also the parameterisation of the control build in Experiment B, so the comparison is against the design being adapted rather than a straw man.

4) Chainlink and Uniswap V3 oracle patterns: *Study:* aggregation of signed reports, TWAP construction and its manipulation cost, freshness handling. *Reuse:* the mathematical patterns. *Modify:* a five-source architecture with an explicit quorum and an ordered rejection sequence, and post-quantum signing of the aggregate, since the aggregate is a durable record.

5) liboqs / PQCclean / ETHFALCON: *Study:* reference implementations, algorithm APIs, published test vectors. *Reuse:* the cryptographic primitives themselves, and the ETHFALCON Solidity verifier for the dispute path and the RQ3 cost measurement. *Explicitly not claimed:* no new post-quantum algorithm, no modification of a primitive's internals. Hand-rolled lattice code would be an unjustifiable risk with no CAVP coverage. The contribution is the protocol integration and its measured cost, not the cryptography.

C. Platform. Four properties are required simultaneously: strong durability for a thirty-year record, EVM equivalence so ERC-3643 and ERC-4626 run unmodified, blocks fast enough for a 60-second liquidation heartbeat, and calldata cheap enough for large post-quantum artefacts. Ethereum L1 fails on cost and latency; a non-EVM L1 would require re-implementing the entire toolchain and has no Falcon verifier; a sidechain rests durability on its own validator set, which is the wrong trade for a record designed to outlive the platform; a permissioned chain lets its operator rewrite the anchor, which defeats the mechanism. An Ethereum L2 rollout provides all four, so the project targets Arbitrum Sepolia (One in production) with USDC as the unit of account and documents on IPFS with the hash on-chain. Nothing in the contracts is Arbitrum-specific: the same bytecode targets any EVM chain, so the choice is about economics and latency, not capability.

The remainder of the stack follows three criteria in order — does it make the security property verifiable, does it survive to production, and does it cost nothing during the project. Solidity 0.8.26 with Foundry and OpenZeppelin 5.x is chosen because the Solidity-native fuzzer and invariant runner are what the verification plan depends on; NestJS on Node 20 with TypeScript gives one language for both ends of the envelope codec, so the canonical byte layout cannot drift; PostgreSQL 16 carries the double-entry ledger where a lost update would be a security bug; Redis 7 Streams with consumer groups gives at-least-once delivery with replay, so a crashed risk worker cannot silently skip a position; and the client is a React 18 PWA with viem and a WASM signer, typed against the ABI so a contract change is a compile error rather than a runtime failure. Everything runs on free tiers, with a documented self-hosted Docker equivalent in the same Compose manifest for every service, so a mid-project tier limit is a configuration change.

D. Module plan and delivery order. A module is admitted only if its interface can be written before its implementation, its dependency set is acyclic, and its acceptance test is runnable by a third party. Each therefore opens with a pull request containing only its interface and invariants, approved by the consumers of that interface before any implementation is written. Delivery is by vertical slice rather than by layer: M0–M4 produce a complete thin path — register, tokenise, deposit, show balance — before lending is touched, so an end-to-end system exists at week 11 rather than week 24 and integration risk is discovered early (Table 7).

Thirteen two-week sprints span FYP-I and FYP-II. Development is trunk-based with feature flags, because with three developers on one contract surface merge debt is the dominant schedule risk; two approvals are required to merge, and contract changes must additionally pass the invariant suite in CI. An architecture decision record accompanies each decision, so the thesis is assembled from artefacts rather than reconstructed at the end. Four gates, each with an exit criterion that re-cuts the plan on failure: **MS1** (week 6) a gold lot registered and tokenised against an attested reserve; **MS2** (week 11, the FYP-I deliverable) the investment path live on test net with envelopes and anchoring,

and a PKR 5,000 deposit yielding a Merkle proof that verifies offline; **MS3** (week 19) lock, borrow, price drop, buffer absorption and liquidation end-to-end with zero bad debt, and at least one liquidation demonstrably prevented that the control build suffers; **MS4** (week 24) every research question answered with measured data.

VI. SECURITY AND FAILURE BEHAVIOUR

The threats have already been named where the mechanisms that answer them were defined, so this section asks the question that a threat list does not: when a component fails or is captured, what does the protocol *do*, and what state is the user left in? Each case is given as failure, detection, response and residual state, and each is an integration test in §VII.

A. Oracle failure or manipulation.

FAILURE	feed down, quorum < 3, deviation > 10%, or a non-monotonic round
DETECTION	at the named check in the pipeline (Model 2), before commit
RESPONSE	no round is committed; once price age passes the asset's heartbeat, borrowing, new locks and liquidation eligibility freeze; repayment and withdrawal stay open
RESIDUAL	positions unchanged; no position can be liquidated on a price the protocol never accepted; the protocol under-acts rather than acting on bad data

B. Post-quantum verification service failure.

FAILURE	the stateless verifier is unavailable or degraded
DETECTION	health check and request timeout; the client sees an explicit refusal, never a silent success
RESPONSE	new authenticated actions stop at the gate; nothing partially executes, because verification precedes every contract call
RESIDUAL	existing on-chain positions remain fully operable directly against the contracts; the service can withhold new authorisations but cannot move funds

C. Anchor batcher failure.

FAILURE	an epoch is not anchored
DETECTION	the epoch counter falls behind the clock; monitored and alerted
RESPONSE	execution is unaffected -- the chain is the system of record -- and the next anchor carries the accumulated leaves
RESIDUAL	commitment is delayed, not lost; the epoch numbering makes the gap explicit and provable rather than hiding it; proofs for the delayed epoch are served once it is written

D. Custodian compromise or false attestation.

FAILURE	an attacker signs an attestation for reserves that do not exist
DETECTION	signature check against the registry key record, sequence monotonicity, and reserve >= intendedSupply, on chain
RESPONSE	the asset cannot reach LIVE; for an already-live asset an overdue or failing re-attestation moves it to FROZEN
RESIDUAL	minting and transfers stop; redemption stays open; the boundary is honest about what it proves -- who signed, and when -- not the physical truth of the reserve

E. Key compromise at any tier.

FAILURE	a Tier-1 service key or a Tier-2 device key is stolen
DETECTION	operational (anomaly, user report); recovery is authorised one tier up, and Tier 0 is offline and hash-based, so a lattice break does not reach it
RESPONSE	validTo is set at the revocation instant; future signatures are rejected at check (2) of the seven-condition verifier
RESIDUAL	every historical signature inside the closed window remains verifiable, because the anchored epoch proves the envelope predates the revocation -- revocation is never destructive

Table 7. Modules, deliverable and the criterion that closes each one. The “done when” column is the acceptance test, not a status report.

Module	Exact stack	Deliverable	Done when
M0 Foundation & devnet	Foundry, Anvil, Docker Compose, GitHub Actions, pnpm workspaces	monorepo, CI pipeline, local chain, deployment scripts	one-command devnet on a clean clone; CI green
M1 PQ core	liboqs 0.12, Rust bindings, wasm-pack, ETHFALCON verifier	signing/verification library, native + WASM	all CAVP vectors pass in both builds; gas report published
M2 PQ-PKI & envelope	PQKeyRegistry, TS/Rust envelope codec, PostgreSQL nonce store	three-tier hierarchy, rotation/revocation, seven-condition verifier	rotation and revocation end-to-end; replay suite rejects at the named check
M3 Registry & compliance	Solidity 0.8.26, OZ 5.x, T-REX, ONCHAINID, Helia IPFS	asset lifecycle, identity registry, compliance module, provider console	provider onboards a gold lot unaided; API-bypassing transfer is refused
M4 Vault accounting	OZ ERC-4626 + virtual offset, Foundry fuzzing, Halmos	share mint/redeem with inflation hardening	10 ⁶ fuzzed sequences: no inflation, no rounding violation
M5 Oracle & proof-of-reserve	Node 20 fetchers (5 APIs), median + 30-min TWAP, ML-DSA-65	signed price rounds, custodian attestation flow	injected malicious source flagged; committed median unmoved
M6 Risk engine	Redis 7 consumer groups, PostgreSQL 16, TypeScript workers	health-factor sweep, notifications, liquidation proposals	p95 < 500 ms at 10,000 positions
M7 Lending & liquidation	LendingPool, CollateralManager, ReserveBuffer, Liquidator, Echidna	index accrual, capacity checks, buffer band, decaying-bonus liquidation	10,000-path Monte Carlo at 3 σ produces zero bad debt
M8 Settlement rail (mock)	NestJS IFiatRail, double-entry ledger, test-net USDC	PKR on/off-ramp simulation with injectable failures and delays	1,000 randomised orders leave the ledger balanced
M9 Anchoring & audit	immutable PQAnchor, Keccak-256 sorted-pair trees, Rust CLI	epoch batcher, proof service, independent verifier	exported bundle verifies on a machine that never contacted us
M10 Clients	React 18, TypeScript, viem, TanStack Query, WASM signer	investor PWA, provider and operations consoles	a non-technical tester completes the full journey unaided
M11 Evaluation harness	Foundry gas snapshots, k6, Python/pandas, ECDSA-only control behind one flag	benchmark suites, simulation datasets, reproduction scripts	every RQ table in §VII populated from measured runs

F. Settlement rail outage.

FAILURE	the mock PKR rail (or, in production, the licensed provider) stalls or rejects
DETECTION	ledger reconciliation and injected-failure handling in IFiatRail
RESPONSE	on- and off-ramping stops at the adapter boundary
RESIDUAL	on-chain positions untouched; the double-entry ledger stays balanced, so no half-settled state exists to reconcile by hand

and key registry are non-upgradeable — an upgradeable audit trail is not an audit trail. The residual exposure in that case is denial of new service, not loss of user funds or of historical evidence.

G. Risk engine outage.

FAILURE	sweep workers die or fall behind
DETECTION	consumer-group lag; at-least-once delivery means a crashed worker’s positions are replayed, not skipped
RESPONSE	liquidations are delayed; borrowers are not notified in time
RESIDUAL	solvency is unaffected in kind, only in latency: any third party can trigger a liquidation, and the chain re-derives HF independently, so the engine’s absence cannot cause an incorrect liquidation -- only a late one

H. Buffer exhaustion and correlated stress.

FAILURE	a market-wide move exhausts the per-epoch absorption cap
DETECTION	the cap test in ReserveBuffer, evaluated before any claim
RESPONSE	further breaching positions take the ordinary liquidation path
RESIDUAL	the protocol degrades exactly to the control build’s behavior; the buffer is an improvement on the baseline, never a dependency of it, which is why exhaustion is a degradation and not an incident

I. Platform-level failures. Two remain, and both are outside the protocol’s control by construction. If the L2 sequencer stalls, no transaction of any kind executes; positions are frozen in place and the design’s response is to inherit L1 durability rather than to add a fallback path that would need its own trust assumptions. If the operator itself is compromised, the 48-hour timelock on parameter changes bounds the damage, pausing is immediate but repayment and withdrawal are never pausable, and the anchor

VII. EXPERIMENTAL MODEL AND EVALUATION

The project makes measurable claims, so the central methodological device is a *control build*: the same application, compiled with the post-quantum authentication fabric of Models 7–9 replaced by ECDSA-only authentication, differing in exactly one dimension because a single build flag selects it. Every figure is reported as a pair, on identical hardware and identical workload. Where a mechanism rather than the cryptography is under test (Experiment B), the control is the same system parameterised as Aave V3 — that is, without the buffer band — so the comparison is against the design being adapted rather than against a straw man.

A. Research questions. RQ1 — what do the primitives cost?

Key generation, signing and verification latency, public-key and signature sizes, for all six primitives, native and in browser WASM, on desktop and mid-range Android. **RQ2 — what does the user feel?** p50/p95 end-to-end latency for each step of the two canonical flows of §II at 1, 10 and 100 concurrent users; the hypothesis is that network round-trips and block confirmation, not lattice arithmetic, dominate perceived latency. **RQ3 — what does the chain pay?** Gas per external call for all eleven contracts in both builds; the anchoring amortisation curve from 1 to 4,096 events per epoch; the empirical break-even against ecrecover;

measured fallback-verifier cost against the ≈ 1.5 M published figure; and projected cost under EIP-8052 pricing. **RQ4 — does the buffer help?** Liquidations avoided, bad debt incurred, peak buffer utilisation and value returned to borrowers, over the 2020 and 2024 gold drawdowns and synthetic GBM paths at 1σ , 2σ and 3σ .

Both hypotheses are pre-registered as falsifiable, and a null result — negligible post-quantum overhead, or a buffer that prevents nothing — is reported as a finding rather than avoided. The contribution is the working system and the measurement design, not a particular sign of the result.

B. Experiment A — cryptographic cost and the break-even point. *Independent variable:* authenticated events per 60-second epoch, $n \in \{1, 2, 4, \dots, 4096\}$. *Conditions:* (i) ECDSA control, (ii) direct on-chain Falcon verification, (iii) post-quantum envelopes with Merkle anchoring. *Measured:* gas per authenticated event, total epoch gas, verification latency, signature and key sizes. *Instrument:* Foundry gas snapshots on a pinned fork, plus the WASM benchmark harness for latency. *Output:* the curve of gas-per-authenticated-event against n for the three conditions, and the empirical n^* at which condition (iii) crosses below condition (i). Model 8 predicts $n^* \approx 14$; the experiment tests that arithmetic against measured snapshots including calldata and storage costs, and a materially different n^* is itself the result. This is the experiment that tests the central architectural claim of the proposal.

C. Experiment B — RWA credit behaviour under transient shocks. *Procedure:* normal price regime, then a short wick below the liquidation point, then recovery; repeated across historical drawdowns and 10,000 Monte Carlo paths. *Conditions:* control (no buffer, Aave-V3 parameters) versus the buffer band of Model 5. *Measured:* liquidations executed, borrower losses, buffer utilisation, bad debt, positions recovered without liquidation. *Hypothesis:* the reserve buffer reduces liquidations caused by transient price shocks without allowing sustained undercollateralisation to persist. The second clause is not decoration — a mechanism that also carries deep, sustained breaches would be a hidden subsidy, and the 3σ paths exist to find the point at which the cap binds before the buffer empties.

D. Experiment C — oracle attack. *Injected:* one, two and three malicious sources; a stale source; a single source with a large deviation; and a correlated majority move inside and outside the TWAP gate. *Measured:* whether the system accepts, rejects, or freezes borrowing — and, critically, *which* of the four named checks produced the outcome. A correct outcome reached at the wrong check counts as a failure, because it means the pipeline order of Model 2 is not doing the work the design attributes to it.

E. Experiment D — protocol invariants. Layered assurance rather than example-based tests, because the contracts hold all the authority. Foundry unit tests at $\geq 90\%$ line and branch coverage; property and invariant fuzzing under Foundry and Echidna with no counterexample in 10^6 call sequences; Halmos symbolic proofs for share arithmetic and health-factor monotonicity, which

Table 8. Measurable targets and how each is checked.

Property	Target	Instrument
Client signing latency	<150 ms p95 in browser WASM	benchmark harness, mid-range Android
End-to-end action latency	<4 s p95 to confirmation	k6 against test net
Risk sweep	<500 ms p95 at 10,000 positions	synthetic position set
Marginal PQ gas cost	<3,000 gas/event at $n \geq 20$	Foundry gas snapshots
Contract test rigour	$\geq 90\%$ branch; 10^6 fuzz sequences	CI gate, blocking merge
Auditability	any action verifiable offline	independent CLI on a clean machine
Recurring infrastructure cost	USD 0 for the project duration	free-tier inventory, self-hosted equivalents documented

examples cannot establish. The invariant set is exactly the one accumulated by §III: attested supply never exceeds reserve; locked collateral never exceeds held collateral; post-borrow $HF \geq 1.15$; `convertToAssets` monotonically non-decreasing; buffer absorption per epoch $\leq \kappa B$; no liquidation while a claim is live; epoch numbers strictly increasing and roots never overwritten; a revoked key cannot authenticate. A hand-written adversarial suite — envelope replay, nonce reuse, cross-domain confusion, expired intent, oracle spoofing, ERC-4626 donation, liquidation front-running, forged proof-of-reserve — asserts rejection at the specific check that catches it, so a passing test proves the mechanism worked rather than that nothing happened.

F. Experiment E — scalability. *Load:* 10,000 open positions with one price tick per second, driven by k6; 10,000 Monte Carlo paths for the risk model. *Measured:* risk-sweep latency (p50/p95), memory, database load, Redis queue throughput and consumer lag, and contract gas under concurrent liquidations. The purpose is to establish that the $O(1)$ interest index of Model 4 and the $O(1)$ anchor of Model 8 actually deliver constant cost in a running system, and to locate the first component whose cost is not constant.

G. Non-functional targets. Table 8 states the numbers the system is required to meet and the instrument that checks each; they are acceptance criteria for the experiments above, not aspirations.

VIII. RESEARCH CONTRIBUTION AND BOUNDARIES

A. What is new, and how each claim is settled. Three families of prior work touch this problem and none combines the two properties the system requires (Table 9). The gap is therefore not a missing primitive but a missing *integration*: an application-layer design that delivers post-quantum durability on an unmodified EVM chain, at a gas cost a retail transaction can bear, while remaining a functioning financial product.

1) Contribution 1 — application-layer post-quantum durability: An end-to-end post-quantum authentication fabric for RWA ownership, in which authenticated actions are verified off-chain

Table 9. Existing approaches and the gap each one leaves.

Family	What it gives, and what it leaves open
Institutional RWA tokenisation platforms	Mature compliance and custody, and the ERC-3643 permissioned-token pattern this project adopts. Minimum investments are institutional, credit against the token is absent or off-platform, and authentication is ECDSA throughout.
Money markets (Aave V3 pattern)	The health-factor, index and liquidation machinery this project reuses. Parameters and the liquidation bonus assume large positions; there is no tolerance band for a transient breach; collateral is crypto-native rather than attested RWA.
Post-quantum work on Ethereum	Verifier implementations and draft precompiles, i.e. primitives. All component-level: none is integrated into a working financial application, and the two precompiles that would make on-chain verification cheap are unshipped drafts.
Migrate the whole chain to PQ signatures	The eventual correct answer, and outside any application's control. It cannot protect records signed in the interim — which is exactly the exposure of an asset tokenised in 2026.

and cryptographically anchored on-chain, rather than requiring native chain-level post-quantum verification. Its novelty is not the primitives but the epoch-anchoring construction of Model 8, which makes the fabric affordable rather than aspirational. *Settled by*: Experiment A — the measured amortisation curve and the empirical break-even n^* against the ECDSA control.

2) *Contribution 2 — collateralised micro-liquidity for tokenised RWA*: Existing RWA tokenisation becomes collateral for a lending mechanism designed around small positions: per-class heartbeats, an origination bound separated from the solvency point, and a liquidation waterfall whose parameters are set for PKR-scale borrowers. *Settled by*: Experiment B's borrower-loss and bad-debt pairs, and the end-to-end journey of MS3.

3) *Contribution 3 — transient-breach protection*: A bounded reserve buffer introduces a state between healthy and liquidatable, allowing temporary price shocks to resolve without immediate liquidation, with a per-epoch absorption cap that keeps the mechanism a first-loss band rather than an unbounded subsidy. *Settled by*: Experiment B at 1σ , 2σ and 3σ , reporting liquidations avoided and bad debt, so the mechanism can fail its own test.

A fourth, methodological, contribution is the control build itself: a reproducible measurement of post-quantum overhead inside a complete financial workflow, isolated to a single compile flag.

B. Sandbox posture: mock money, production-grade code. A student team cannot lawfully hold client funds, custody bullion or issue securities, and the project does not attempt to. It is nonetheless built to production standards, and the two facts are reconciled by one rule: every real-world dependency is reached only through an interface, and the prototype ships a simulated implementation of that interface. Four such seams exist — IFiatRail (mock PKR rail with a full double-entry ledger), ICustodianAdapter (scripted reserve schedule signing real

Table 10. Risks and the response decided in advance, so that a schedule shock does not become a design shock.

Risk	Pre-committed response
Browser WASM signing too slow or too large	fall back to a server-side signer with an explicit trust note; the envelope format is unchanged, so nothing else moves
Fallback verifier exceeds the block gas limit	off-chain verification with an on-chain challenge window; the anchor already provides the commitment it disputes
Public price APIs rate-limit or disappear	five independent sources with a three-source quorum; a recorded historical feed drives evaluation runs
Scope overrun across four asset classes	gold is the only mandatory class; the registry is asset-agnostic
Free-tier limits reached mid-project	every service has a documented self-hosted Docker equivalent in the same Compose manifest
Copyright licence blocks a production path	clean-room implementation of the same ERC-3643 interface; contracts depend on the interface, not the implementation
Merge debt across three developers on one contract surface	trunk-based development with feature flags, two approvals, and an invariant suite blocking merge

ML-DSA-65 attestations), IPriceOracle (five live public price APIs, aggregated and signed by us, standing in for a commercial feed) and IIdentityRegistry (a self-hosted ERC-3643 claim issuer standing in for a KYC vendor). The settlement asset is a valueless test-net stablecoin. Consequently the mock is *only* the counterparty: the cryptography, the contracts, the accounting, the risk engine and the tests are the real thing, and replacing a stub with a licensed provider is a configuration change plus that provider's integration tests.

C. Regulatory stance. Pakistan's Virtual Assets Act 2026 empowers PVARA to license virtual-asset service providers. The prototype performs no licensable activity: no client funds, no third-party custody, no public offering, and a valueless test-net token. Two design decisions nevertheless pre-position it for a future licence — the adapter boundary above, and ERC-3643, which enforces transfer restrictions inside the token rather than in the application, so compliance cannot be bypassed by calling the contract directly.

D. Boundaries. Out of scope, deliberately: real custody, securities issuance, main-net deployment, production AML screening, a secondary trading venue or order book, mobile-native applications, and a formally audited production release. Gold is the only mandatory asset class; T-bills, Sukuk and fractional REITs are stretch goals, and because the registry is asset-agnostic, cutting them costs no rework. Three exposures are acknowledged rather than solved and are stated in §IV: a custodian who signs a false but well-formed attestation, collusion among a majority of price sources, and the operational trust in the identity process that a device-key recovery flow requires.

E. Principal risks and pre-committed responses.

F. Deliverables. Verified contracts on Arbitrum Sepolia; the investor PWA with provider and operations consoles; a post-quantum signing library with a published NIST CAVP confor-

mance report; the independent offline audit-verifier CLI; the mock settlement rail with its double-entry ledger; the evaluation dataset, the control build and the reproduction scripts; and the thesis with its ADR log, threat model and API specification. Taken together these constitute both a working sandbox system and the evidence for the claim the title makes — that post-quantum durability is affordable at the application layer today.

APPENDIX A — SYMBOLS AND TERMS

A. Symbols

A, S	total assets held by a vault, in USDC, and total shares outstanding
$\sigma = 10^6$	virtual-share offset added to S (with 1 added to A) to defeat the ERC-4626 inflation attack
D	outstanding debt of a position: a scaled balance multiplied by the global borrow index
q_i, p_i	quantity of collateral asset i locked, and its last committed oracle price
θ_i	maximum loan-to-value: the fraction of asset i 's value that may be borrowed at origination
λ_i	liquidation threshold, $\lambda_i > \theta_i$
LTV, HF	loan-to-value $D / \sum_i q_i p_i$; health factor $\sum_i q_i p_i \lambda_i / D$, where $HF < 1$ means undercollateralised
U, U^*	pool utilisation and its optimal level (the kink), $U^* = 80\%$
r_0, s_1, s_2	base rate and the two interest-curve slopes below and above the kink (0%, 4%, 75%)
I_t, b_u	global borrow index at time t , and a borrower's scaled balance
B, κ	reserve buffer balance, and the per-epoch absorption cap $\kappa = 0.40$
$\phi, \beta(t)$	liquidation close factor (50%) and the bonus decaying over 20 blocks
δ	oracle deviation threshold against the 30-minute TWAP, $\delta = 10\%$
e, n	anchoring epoch number, and the number of envelope leaves committed in that epoch
G_a, G_p, G_{ec}	gas for an epoch anchor, a depth-12 inclusion proof, and <code>ecrecover</code>
$1\sigma\text{--}3\sigma$	price-shock magnitudes in standard deviations, used for the stress paths of §VII

B. Terms

Anchor, epoch, leaf	an <i>anchor</i> is one Merkle root written on-chain per 60-second <i>epoch</i> ; a <i>leaf</i> is the hash of one signed envelope included in it
PQEnvelope	the canonical structure every post-quantum signature covers (Model 7)
Domain separator	hash binding an intent to one chain, verifying contract and action type, so it cannot be reused elsewhere
Nonce, TTL	strictly incrementing counter per signer and action; time-to-live bounding how long an intent stays valid
HNDL	harvest-now-decrypt-later: store signatures or ciphertext today, break them once a quantum computer exists
ECDSA, ECDLP, Shor	elliptic-curve signatures, the discrete-logarithm problem they rest on, and the quantum algorithm that solves it in polynomial time
ML-KEM, ML-DSA, SLH-DSA	NIST-standardised post-quantum key encapsulation (FIPS 203), lattice signatures (FIPS 204) and hash-based signatures (FIPS 205)
Falcon	NTRU-lattice signature scheme with very compact signatures, selected by NIST for standardisation as FN-DSA
Security category	NIST classification of parameter sets by attack effort: category 2 $\approx$ AES-128, category 3 $\approx$ AES-192
CAVP	NIST Cryptographic Algorithm Validation Program: published test vectors used here as M1's conformance gate
ERC-3643, ERC-4626	permissioned-token standard enforcing transfer restrictions inside the token; tokenised-vault standard for share accounting
Inflation (donation) attack	manipulating a near-empty ERC-4626 vault's share price so later small deposits round to zero shares
Scaled balance, index	debt stored as a normalised amount times a global compounding index, so interest accrues with no per-user loop
Close factor, bonus	maximum fraction of a position liquidatable in one call (50%), and the discount paid to the liquidator (3–10% by asset)
ReserveBuffer	protocol-funded reserve absorbing shallow, transient HF breaches within a 40%-per-epoch cap
TWAP	time-weighted average price over 30 minutes; the deviation reference for a committed median
Proof-of-reserve	custodian-signed attestation that a stated reserve exists; required for an asset to reach LIVE
MEV	value extracted by reordering transactions, e.g. racing a liquidation for its bonus
front-running	upgradeable-proxy pattern; granted here to the pool, collateral manager and oracle, and deliberately withheld from the anchor and key registry
UUPS	testing that a stated property holds over random call sequences rather than fixed examples
Invariant / property	proving a property for all inputs by constraint solving rather than sampling
fuzzing	the same application with post-quantum authentication replaced by ECDSA behind one build flag — the measurement baseline of §VII
Symbolic execution	WebAssembly (native-speed in-browser signing), progressive web app, and Pakistan's Virtual Assets Regulatory Authority
Control build	
WASM, PWA, PVARA	

REFERENCES

REFERENCES

- [1] NIST, *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard* (ML-KEM), Aug. 2024.
- [2] NIST, *FIPS 204: Module-Lattice-Based Digital Signature Standard* (ML-DSA), Aug. 2024.

- [3] NIST, *FIPS 205: Stateless Hash-Based Digital Signature Standard* (SLH-DSA), Aug. 2024.
- [4] P.-A. Fouque *et al.*, “Falcon: fast-Fourier lattice-based compact signatures over NTRU,” NIST PQC submission specification.
- [5] PQ-CRYSTALS project, “CRYSTALS-Kyber and CRYSTALS-Dilithium: specifications and reference implementations.”
- [6] D. J. Bernstein *et al.*, “SPHINCS+: stateless hash-based signatures — specification and reference code.”
- [7] Open Quantum Safe project, *liboqs* — open-source C library of post-quantum KEMs and signatures (MIT licence).
- [8] PQClean project, *PQClean* — portable reference implementations of PQC schemes.
- [9] NIST, *Cryptographic Algorithm Validation Program (CAVP)* — ML-KEM, ML-DSA and SLH-DSA test vectors.
- [10] P. W. Shor, “Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer,” *SIAM J. Comput.*, vol. 26, no. 5, 1997.
- [11] NSA, *Commercial National Security Algorithm Suite 2.0 FAQ* — harvest-now-decrypt-later rationale and migration timelines.
- [12] IETF TLS working group, “Hybrid key exchange in TLS 1.3” and the X25519MLKEM768 group, Internet-Drafts.
- [13] “EIP-8051: Precompile for ML-DSA signature verification,” Ethereum Improvement Proposals, draft.
- [14] “EIP-8052: Precompile for Falcon signature verification,” Ethereum Improvement Proposals, draft.
- [15] “EIP-7932: Algorithmic transaction container” (signature agility), Ethereum Improvement Proposals, draft.
- [16] ZKNox, *ETHFALCON* — Falcon variants and Solidity verifier for the EVM, with gas benchmarks.
- [17] Ethereum Foundation, “Post-quantum cryptography and Ethereum’s future-proofing roadmap,” 2025.
- [18] ERC-3643, “T-REX: Token for Regulated EXchanges,” Ethereum Improvement Proposals.
- [19] Tokeny Solutions, *T-REX* — ERC-3643 reference implementation (GPL-3.0).
- [20] ONCHAINID, *solidity* — on-chain identity and claim contracts used by ERC-3643.
- [21] ERC-4626, “Tokenized Vault Standard,” Ethereum Improvement Proposals.
- [22] OpenZeppelin, *openzeppelin-contracts* v5.x — audited ERC-20/ERC-4626, access-control and proxy libraries (MIT).
- [23] EIP-1822 (universal upgradeable proxy standard) and EIP-1967 (standard proxy storage slots).
- [24] Aave, *Aave V3 Technical Paper*, 2022 — health factor, liquidation mechanics, interest-rate strategy.
- [25] Aave, *aave-v3-core* — protocol implementation and risk-parameter configuration.
- [26] R. Leshner and G. Hayes, *Compound: The Money Market Protocol*, whitepaper, 2019 — utilisation-based rate model.
- [27] L. Breidenbach *et al.*, *Chainlink 2.0: Next Steps in the Evolution of Decentralized Oracle Networks*, 2021.
- [28] H. Adams *et al.*, *Uniswap v3 Core*, 2021 — time-weighted average price oracles and manipulation cost.
- [29] MakerDAO, *median* and the Oracle Security Module — median aggregation of signed price messages.
- [30] R. C. Merkle, “Protocols for public key cryptosystems,” *Proc. IEEE Symp. Security and Privacy*, 1980.
- [31] B. Laurie, A. Langley and E. Kasper, *Certificate Transparency*, RFC 6962, IETF, 2013 — append-only logs and inclusion proofs.
- [32] OpenZeppelin, *MerkleProof* — sorted-pair Keccak-256 proof verification in Solidity.
- [33] OpenZeppelin, “ERC-4626 inflation attack and the virtual-shares mitigation,” contracts documentation and security advisory.
- [34] Trail of Bits, *Echidna* — property-based fuzzer for EVM smart contracts.
- [35] a16z, *Halmos* — symbolic bounded model checker for Solidity.
- [36] Foundry, *Foundry Book* — fuzzing, invariant testing and gas snapshots.
- [37] Trail of Bits, *Building Secure Contracts* — invariant-driven development guidance.
- [38] Consensys Diligence, *Smart Contract Best Practices* and the SWC Registry of known weaknesses.
- [39] P. Daian *et al.*, “Flash Boys 2.0: frontrunning, transaction reordering, and consensus instability in decentralized exchanges,” *IEEE Symp. Security and Privacy*, 2020.
- [40] Offchain Labs, *Arbitrum Nitro* whitepaper and developer documentation.
- [41] IPFS project, *Helia* — JavaScript IPFS implementation and content-addressing specifications.
- [42] Government of Pakistan, *Virtual Assets Act 2026*, and PVARA draft *Virtual Asset Services Regulations 2026* (public consultation).
- [43] NIST, *SP 800-90A Rev. 1: Recommendation for Random Number Generation Using Deterministic Random Bit Generators*.